

Why Does Multilingual Training Improve Theorem Proving? A Small-Scale Controlled Study

Amadeo De La Vega

Department of Computer Science
University of Maryland

Mehak Gupta

Department of Computer Science
University of Maryland

Huan Zhou

Department of Civil and Environmental Engineering
University of Maryland
College Park, MD 20742, USA
{adelaveg,mgupta25,hzhou715}@umd.edu

Abstract

Multilingual theorem-proving models can outperform monolingual baselines, but the reason is unclear. Do models learn proof patterns shared by Lean and Rocq, or do they mainly benefit from seeing more syntax variation during training? We study this question with three controlled CodeT5-small models in the ProofWala setting: E1, a Lean-only baseline; E3, a real multilingual Lean+Rocq model; and E4, a pseudo-multilingual Lean model built with meaning-preserving syntax changes. We evaluate all models with the same proof-search pipeline on two Lean benchmarks. On **Easy_Lean**, E4 performs best (63.6% pass rate, versus 38.6% for E1 and 30.5% for E3). On **Hard_Lean**, E3 performs best (4.9%, versus 2.6% for E1 and 3.4% for E4). The main finding is distribution-dependent: syntax variation helps most on **Easy_Lean**, while real multilingual training helps most on **Hard_Lean**.

1 Problem Statement, Motivation, and Approach

Recent neural theorem provers typically train a language model to map a proof state to the next tactic, then use the model inside a search procedure. ProofWala reports that multilingual training across Lean and Rocq improves proof-search success (Thakur et al., 2025). The open question is why. We study two hypotheses. **H1, cross-language transfer**, says that models learn proof patterns that transfer across proof assistants. **H2, regularization**, says that syntax variation improves robustness by discouraging overfitting to language-specific syntax.

This project asks:

When does multilingual training improve proof search, and is the gain due to cross-system transfer or regularization?

We compare three models that differ only in training data. E1 is trained on Lean proof steps only. E3 is trained on real multilingual Lean+Rocq data. E4 is trained on Lean plus a pseudo-Lean variant created by consistently renaming local variables and hypotheses. E4 is the key control: it adds syntax variation without adding Rocq syntax, tactics, or libraries. Thus, E3 versus E4 separates genuine cross-system transfer from syntax-variation regularization.

We evaluate on two Lean proof-search benchmarks. **Easy_Lean** is a controlled in-project Lean benchmark with relatively regular theorem patterns. **Hard_Lean** is the combined miniF2F-based external benchmark, which is more diverse and harder. This naming follows the project presentation and emphasizes the role of benchmark difficulty and distribution.

Our approach measures both final proof success and search behavior. The main metric is pass rate: the fraction of attempted theorems for which the search procedure finds a complete proof. We also record accepted tactic rates, proof-tree size, branching factor, and search time. These diagnostics help determine whether a model succeeds because it generates more useful tactics, searches more efficiently, or is simply better aligned with a benchmark distribution.

The contribution is a small but controlled pilot study. Rather than claiming a universal winner, we show that the observed mechanism depends on the evaluation distribution: pseudo-multilingual regularization dominates on Easy.Lean, while real multilingual training is strongest on Hard.Lean.

2 Related Work and Context

Neural theorem proving commonly combines tactic prediction with proof search. GPT-f showed that generative language models can guide formal proof construction in Metamath (Polu & Sutskever, 2020). Later systems extended this paradigm to interactive theorem provers, where a model proposes tactics conditioned on a proof state and a verifier checks each step (Huang et al., 2019; Yang & Deng, 2019; Blaauwbroek et al., 2020; Yang et al., 2023).

Our study builds most directly on ProofWala, which provides a multilingual data and proof-search pipeline for Lean and Rocq (Thakur et al., 2025). ProofWala reports that mixed-language training improves proof-search success over monolingual baselines, but it does not isolate why. A central novelty of our project is to separate two explanations that are usually grouped together: cross-system transfer and syntax-variation regularization. If Lean and Rocq share useful proof-state transitions, the real multilingual model should help most. If the main benefit is exposure to more syntax variation, then a pseudo-multilingual control should recover much of the gain without using Rocq examples.

This question is related to the distinction between syntax and semantics in AI. Chomsky’s formal-language tradition emphasized that grammatical form can be studied separately from meaning (Chomsky, 1957), while connectionist work associated with Hinton argues that neural networks can learn useful internal representations from data (Rumelhart et al., 1986). Later debates make the same point in modern terms: statistical models can find patterns in form, but form alone may not guarantee grounded meaning (Norvig, 2011; Bender & Koller, 2020). In theorem proving, “semantics” has a concrete meaning: a tactic changes the proof state. This is an operational view of meaning, following structural operational semantics (Plotkin, 1981). H1 predicts transfer at this level: a model trained on one proof language learns state-change patterns that help in another. H2 predicts a syntactic effect: more syntax variation prevents overfitting to Lean-specific syntax without requiring true cross-system transfer.

Also, this distinction connects to two adjacent lines of work. First, diversity-driven proof search methods such as 3D-Prover argue that theorem proving benefits from exploring non-redundant candidate tactics rather than repeatedly sampling similar actions (Lamont et al., 2025). Second, data augmentation work such as Lean4Trace shows that additional tactic and state variation can improve Lean theorem proving (Nesterov et al., 2024). Our E4 condition adapts this intuition into a causal control: it adds syntax variation while holding the proof assistant fixed.

Finally, miniF2F provides an external benchmark for formal Olympiad-style mathematics across proof assistants (Zheng et al., 2022). We use miniF2F-derived Lean theorems as Hard.Lean to test whether conclusions from a controlled Lean benchmark generalize to a harder theorem distribution.

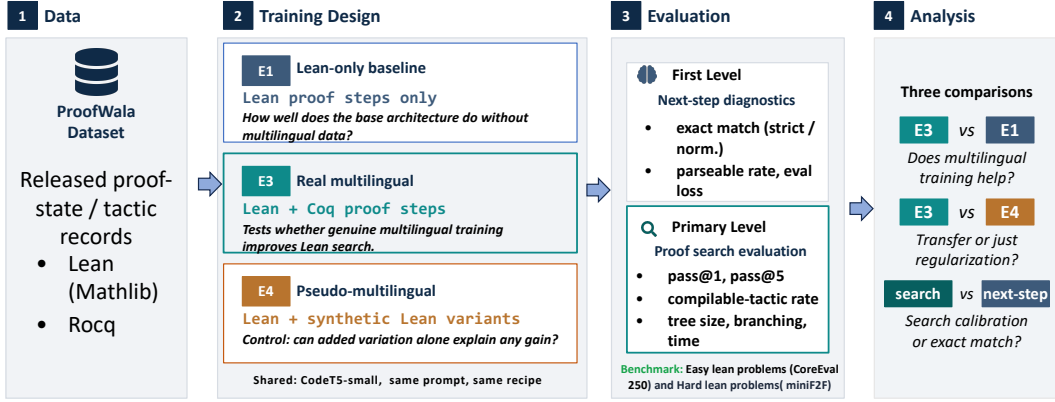


Figure 1: Methodology of Three-Condition Controlled Pilot for Multilingual Theorem Proving.

Table 1: Main experimental conditions. All three trained models use the same base model, tokenizer, prompt grammar, and Lean evaluation pipeline.

ID	Name	Purpose
E1	Lean-only baseline	Trained only on Lean proof steps. This model shows how well the chosen architecture performs without multilingual data.
E3	Real multilingual model	Trained on ProofWala multilingual data containing Lean and Rocq proof steps. This model tests whether real multilingual training improves Lean proof search.
E4	Pseudo-multilingual control	Trained on Lean proof steps plus a synthetic Lean variant. This model tests whether syntax variation alone can explain any gain.

3 Methodology

3.1 Task overview

In ProofWala, the model repeatedly observes a *proof state*—the current assumptions and unsolved goals—and proposes a *tactic*, a short command that asks Lean or Rocq to transform the state. The proof assistant checks each tactic, and a proof succeeds only when all goals are closed.

We evaluate two abilities. First, next-step diagnostics test whether a model can imitate a held-out tactic. Second, proof search tests whether the model can actually finish unseen Lean theorems. Proof search is the main end task because generated tactics must be accepted by Lean, not merely look plausible.

Figure 1 summarizes the pipeline. We use the released ProofWala dataset, freeze evaluation references, train E1/E3/E4 from the same CodeT5-small base model, and evaluate them with the same Lean proof-search pipeline on EasyLean and HardLean.

3.2 Experimental design

We train three models with the same architecture and prompt format, varying only the training data (Table 1). This design separates the two hypotheses: E3 tests real Lean+Rocq transfer, while E4 tests whether syntax variation alone can explain the gain.

If E3 beats both E1 and E4, the result favors H1. If E4 matches or exceeds E3, the result favors H2. We also examine search diagnostics to see whether differences come from more useful candidate tactics during search.

3.3 Data sources and splits

We use the released `ami tayusht/ProofWalaDataset` rather than regenerating proof-step data. E1 uses the Lean training split. E3 uses a project-compatible multilingual split with Lean and Rocq examples. E4 uses Lean data plus a synthetic Lean variant and deliberately contains no real Rocq examples. All evaluation references are frozen before training and reused across E1, E3, and E4.

3.4 Model and training setup

All three models start from `Salesforce/codet5-small`. This is a smaller, but similar model used in the ProofWala experiments (`Salesforce/codet5-base`). We chose this smaller version to reduce the training time. The input is the serialized proof state and the output is one tactic. We keep the same tokenizer, prompt format, sequence lengths, and training recipe where possible. Our final pilot models were trained with one RTX A5000 GPU and a 12-hour wall-time budget on the UMD Nexus cluster.

3.5 Pseudo-multilingual control construction

We designed the E4 condition to answer a simple question: can the benefit of multilingual training be explained by syntax variation alone? To test this, we augment Lean training data with synthetic Lean examples that look syntactically different but preserve the same proof structure. The synthetic examples are not translated into Rocq and do not contain Rocq tactics, Rocq lemmas, or Rocq-specific syntax.

We keep the transformation conservative. Our script identifies local variables and hypotheses, renames them to placeholders such as `pseudo_0`, and applies the same rename map to the hypotheses, goals, and target tactic. We avoid names with likely global meaning, such as Lean keywords, namespaces, theorem names, constructors, tactic names, and qualified names such as `Nat.add`. Thus E4 changes syntax but not the proof language, task, or prompt format.

Because synthetic data can introduce subtle errors, E4 is a pilot control rather than a final causal control. We audit generated examples before training and prefer dropping risky transformations to training on invalid proof states.

4 Evaluation

We evaluate each model at two levels (Table 2). Next-step diagnostics ask whether the model can predict a held-out tactic from a proof state. Proof-search evaluation places the model inside the ProofWala search loop (Thakur et al., 2025): the model proposes tactics, Lean checks them, and search continues until a proof is found or the budget is exhausted. Proof search is the primary evaluation because it measures whether the model can finish proofs, not only imitate recorded steps.

4.1 Benchmarks and metrics

We evaluate the models on two Lean benchmarks. The first is **Easy Lean**, a controlled set of 250 Lean problems that we constructed for this study. The problems are relatively regular and share many proof-step patterns, so **Easy Lean** tests performance near the Lean training distribution.

The second is **Hard Lean**, our combined miniF2F-based external benchmark. miniF2F is a standard theorem-proving benchmark containing formal Olympiad-level mathematics problems across proof assistants such as Lean, Metamath, Isabelle, and HOL Light (Zheng

Table 2: Two-level evaluation design and metric meanings.

Evaluation Level	Metric	Meaning
Next-step diagnostics (Huang et al., 2019; Yang & Deng, 2019; Blaauwbroek et al., 2020)	Exact match	Measures whether the model predicts the same next tactic as the reference proof.
	Normalized exact match	Measures tactic prediction after removing minor formatting differences.
	Parseable output rate	Measures whether the generated tactic is syntactically valid Lean code.
	Mean evaluation loss	Measures prediction error on held-out proof-step data.
Proof-search evaluation (Thakur et al., 2025; Yang et al., 2023; Lamplé et al., 2022)	Pass@1	Measures whether the theorem is proved using the model’s top-ranked choices.
	Pass@ k	Measures whether proof search succeeds after exploring up to k candidate tactics.
	Accepted-tactic rate	Measures how often generated tactics are accepted by Lean during search.
	Early dead-end rate	Measures how often search stops early because no useful tactic is found.
	Average search time	Measures the computational cost of attempting each theorem.

et al., 2022). Hard Lean is harder and more diverse than Easy Lean, so it tests external generalization rather than in-project benchmark fit.

The main proof-search metric is pass rate: the fraction of attempted theorems that are proved. We also report accepted tactic rate, proof-tree size, branching factor, early dead-end rate, and average search time to understand how each model searches.

4.2 Handling timeouts and scope of claims

Proof search is computationally expensive, so a 12-hour Nexus job may not finish every theorem for every model. We therefore report both the full attempted set and an *intersection view* restricted to theorem names attempted by all three models.

Our claims are limited to this pilot setting. The models are CodeT5-small checkpoints trained under class-cluster storage and wall-time constraints, so their absolute pass rates should not be interpreted as final performance numbers. The main evidence is the relative ordering and search behavior of E1, E3, and E4 under the same model size, data-processing pipeline, benchmark, and search budget. Because each experiment uses a single seed, we treat the results as pilot evidence rather than statistically confirmed effects.

5 Results and Analysis

Table 3 gives the main proof-search results. We report both the full attempted set and the theorem-name intersection attempted by all three models. The intersection view is the cleaner comparison when jobs time out at different points.

Table 3: Proof-search results on Easy Lean and Hard Lean. Pass rate is proved / attempted.

Benchmark	Model	Attempted	Proved	Pass Rate	Intersection Proved	Intersection Pass Rate
Easy Lean	E1: Lean-only	127	49	38.6%	49	38.6%
	E3: Real multilingual	141	43	30.5%	39	30.7%
	E4: Pseudo-multilingual	198	126	63.6%	78	61.4%
Hard Lean	E1: Lean-only	77	2	2.6%	2	2.6%
	E3: Real multilingual	81	4	4.9%	4	5.2%
	E4: Pseudo-multilingual	87	3	3.4%	3	3.9%

The results are distribution-dependent. On Easy_Lean, E4 is the clear winner: it proves 126 theorems overall and 78 on the shared intersection, compared with 49 for E1 and 39 for E3. The Easy_Lean result was unexpected and surprising for two reasons. First, ProofWala suggests that real multilingual training should improve over a Lean-only baseline, but E3 is below E1 here. Second, E4 improves by a large margin over both E1 and E3. This supports the regularization hypothesis, but we do not yet understand why the improvement is so strong; explaining its size requires further experiments and a more careful audit of the pseudo-Lean transformation.

Hard_Lean changes the ordering. E3 proves 4 theorems, compared with 3 for E4 and 2 for E1. The pass rates are low because Hard_Lean is timeout-heavy, but the ordering matters: real multilingual training is strongest on the harder external benchmark, while E1 and E4 become much closer than on Easy_Lean.

Table 4: Search diagnostics. Accepted action rate is parser/action acceptance during proof search, not a standalone proof-correctness metric.

Benchmark	Model	Valid Tactics / State	Accepted Rate	Early Dead-End	Mean Nodes	Mean Branching	Mean Time (s)
Easy_Lean	E1	12.060	1.000	0.000	3.141	4.164	231.338
	E3	12.840	1.000	0.000	2.915	3.258	195.715
	E4	12.789	1.000	0.000	3.232	3.929	150.511
Hard_Lean	E1	11.358	1.000	0.000	2.117	2.137	490.195
	E3	12.941	1.000	0.000	2.185	1.981	456.315
	E4	11.781	1.000	0.000	2.195	2.298	405.301

The search diagnostics reinforce the main story. E4 is fastest on both benchmarks, which helps explain its Easy_Lean throughput. E3 generates the most valid tactics per state on Hard_Lean and solves the most Hard_Lean theorems, suggesting that its benefit appears most clearly on the harder external distribution.

Table 5: Hard_Lean theorems proved by each model.

Model	Theorem	Time (s)	Tactic
E1	mathd.numbertheory_342	122.1	norm_num1
E1	mathd.algebra_176	57.4	ring
E3	amc12b_2020_p2	114.8	ring1
E3	mathd.numbertheory_207	100.7	norm_num
E3	mathd.algebra_329	68.6	linarith
E3	mathd.algebra_176	85.7	ring
E4	mathd.numbertheory_207	114.6	ring
E4	mathd.algebra_176	86.2	ring1
E4	mathd.numbertheory_175	92.1	norm_num

The overlap is also informative. All three models prove mathd.algebra_176; E3 and E4 both prove mathd.numbertheory_207; and each model has at least one unique success. Thus Hard_Lean does not merely repeat the Easy_Lean ranking.

5.1 Interpretation and Limitations

The strongest conclusion is not that one training method is always best. The mechanism depends on the theorem distribution. Easy_Lean supports H2: syntax variation improves Lean-like proof search. Hard_Lean gives weaker but useful support for H1: real multilingual training can help on harder external theorem styles.

Several limitations qualify these claims. E3 used a practical multilingual split rather than a perfectly token-matched Lean+Rocq mixture. E4’s pseudo-Lean transformation is conservative but still requires further auditing. Hard_Lean has low proof counts, so its ordering should be treated as pilot evidence. Finally, all experiments use one model size, one seed, and Nexus class-cluster time and storage constraints. Figure 2 summarizes the final interpretation.

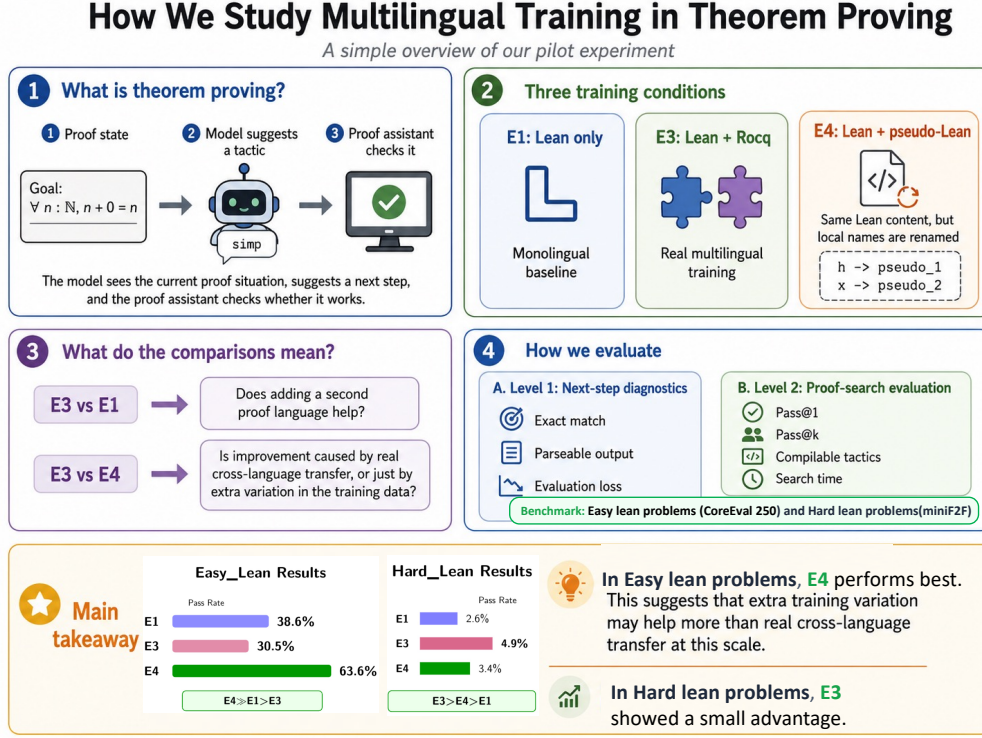


Figure 2: Summary of the main result: Easy_Learn favors pseudo-multilingual regularization, while Hard_Learn gives the strongest result to real multilingual training.

6 Data, Code, and Reproducibility

All code, configs, and frozen data splits are available at <https://github.com/AmadeoDeLaVega/multilingual-project>. The repository contains the ProofWala and ITP-interface code, Hydra training configs for E1/E3/E4, frozen split references, SLURM scripts under scripts/nexus/, and the copied evaluation outputs that produced Table 3. The Salesforce/codet5-small base weights are downloaded from Hugging Face at install time and are not bundled; the Nexus SLURM scripts assume the UMIACS class partition and will need adaptation for other clusters. Each training run used one NVIDIA A5000 GPU with a 12-hour wall-time budget. Proof-search evaluation was also run through SLURM and may stop by wall-time, which is why we report both attempted-theorem counts and intersection results. Re-running with the same configs and seed 42 should reproduce Table 3 up to normal CUDA and cluster nondeterminism. See README.md for step-by-step setup instructions.

7 Statement of Contributions

Amadeo De La Vega led the development of the project, the implementation of the training and proof-search pipeline, ran the main Nexus jobs, managed result collection, and revised the report and presentation. Mehak Gupta contributed to code review and Nexus testing, helped write the problem statement, related work, and results discussion, and prepared the presentation slides. Huan Zhou contributed to code review and Nexus testing and helped write the methodology and evaluation-design sections. All authors discussed the experimental framing and interpretation.

References

- Emily M. Bender and Alexander Koller. Climbing towards NLU: On meaning, form, and understanding in the age of data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 5185–5198, 2020. doi: 10.18653/v1/2020.acl-main.463.
- Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. Tactic learning and proving for the Coq proof assistant. In *LPAR 2020: 23rd International Conference on Logic for Programming, Artificial Intelligence and Reasoning*, volume 73 of *EPiC Series in Computing*, pp. 138–150. EasyChair, 2020. doi: 10.29007/wg1q. URL <https://arxiv.org/abs/2003.09140>.
- Noam Chomsky. *Syntactic Structures*. Mouton, 1957.
- Daniel Huang, Prafulla Dhariwal, Dawn Song, and Ilya Sutskever. GamePad: A learning environment for theorem proving. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1806.00608>.
- Sean Lamont, Bo Liu, Lars Buesing, Anima Anandkumar, and Arthur Gretton. 3D-Prover: Diversity driven theorem proving with determinantal point processes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2410.11133>.
- Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Aurélien Rodriguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2205.11491>.
- Vasili Nesterov, Yermek Kapushev, and Mikhail Burtsev. Lean4trace: Data augmentation for neural theorem proving in Lean. In *ICML 2024 Workshop on AI for Math (AI4MATH)*, 2024. URL <https://openreview.net/forum?id=sjLWmLeJ6R>.
- Peter Norvig. On Chomsky and the two cultures of statistical learning, 2011. URL <https://norvig.com/chomsky.html>.
- Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Aarhus University, 1981. URL <https://www.cs.cmu.edu/~crary/819-f09/Plotkin81.pdf>.
- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020. URL <https://arxiv.org/abs/2009.03393>.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Amitayush Thakur, George Tsoukalas, Greg Durrett, and Swarat Chaudhuri. ProofWala: A framework for multilingual proof data synthesis and theorem-proving. In *2nd AI for MATH Workshop at ICML 2025*, 2025. URL <https://arxiv.org/abs/2502.04671>.
- Kaiyu Yang and Jia Deng. Learning to prove theorems via interacting with proof assistants. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6984–6994, 2019. URL <https://arxiv.org/abs/1905.09381>.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.15626>.
- Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2109.00110>.